

Week 10 - Draft Report

Group Name: Group 03

Member: Raphael Carvalho de Figueiredo Rocha

Chosen Topic: Organização e priorização de tarefas no planejamento em DevOps

Organization and Prioritization of Tasks in DevOps Planning

Introduction

DevOps é frequentemente associado à automação, à integração contínua, à entrega contínua e à aceleração do ciclo de deploy. No entanto, esses elementos técnicos, embora importantes, não explicam sozinhos o bom funcionamento de um ambiente DevOps. Para que o fluxo seja realmente eficiente, também é necessário que o trabalho seja bem organizado desde o início, que as tarefas sejam corretamente priorizadas e que exista alinhamento entre os diferentes envolvidos no processo.

Neste contexto, o presente trabalho tem como foco a organização e a priorização de tarefas no planejamento em DevOps. Esse tema é relevante porque ambientes DevOps operam com ciclos curtos, mudanças frequentes e necessidade constante de adaptação. Quando o planejamento é fraco, surgem atrasos, retrabalho, falhas de comunicação, conflitos entre equipes e perda de produtividade. Em contrapartida, quando requisitos, prioridades e tarefas são bem estruturados, as práticas DevOps podem ser executadas com maior continuidade, previsibilidade e eficiência.

O objetivo deste relatório é analisar, com base na literatura selecionada, como o planejamento em DevOps é influenciado por requisitos, práticas de colaboração, automação, ferramentas e mecanismos de feedback contínuo. A proposta é mostrar que a organização e a priorização das tarefas não são elementos periféricos, mas parte essencial da lógica de funcionamento de DevOps.

Literature review

A literatura analisada mostra que DevOps não deve ser entendido apenas como um pipeline técnico de desenvolvimento e entrega, mas como uma combinação de elementos organizacionais, gerenciais e operacionais.

O estudo *Requirements management in DevOps environments: a multivocal mapping study* destaca que DevOps ainda precisa amadurecer em aspectos como reuso de requisitos, comunicação entre áreas e stakeholders, rastreabilidade, requisitos não funcionais e integração de ferramentas de requisitos à toolchain DevOps. Isso é

especialmente importante para o tema deste trabalho, pois a organização e a priorização das tarefas dependem diretamente da clareza dos requisitos e da visibilidade de suas dependências.

O artigo *DevOps Project Management Tools for Sprint Planning, Estimation and Execution Maturity* contribui de forma mais direta para o foco do trabalho, pois trata de ferramentas e mecanismos ligados ao planejamento de sprint, estimativa de esforço, tomada de decisão e acompanhamento da maturidade da execução. Esse estudo reforça que, em ambientes DevOps, o planejamento precisa ser contínuo, adaptável e apoiado por instrumentos que reduzam incertezas e ajudem na definição de prioridades.

O artigo *DevOps Capabilities, Practices, and Challenges: Insights from a Case Study* apresenta uma perspectiva aplicada, baseada em estudo de caso real. Ele mostra como práticas como automação de pipeline, monitoramento, continuous delivery e colaboração entre equipes cross-functional se manifestam na prática. Esse estudo é relevante porque liga planejamento e execução, mostrando que decisões tomadas no início do fluxo afetam o restante do processo.

O artigo *Building a Collaborative Culture: A Grounded Theory of Well Succeeded DevOps Adoption in Practice* enfatiza que o núcleo da adoção bem-sucedida de DevOps é a construção de uma cultura colaborativa entre desenvolvimento e operações. A automação aparece como meio importante, mas não como objetivo principal. Esse ponto é central para este trabalho, pois organização e priorização dependem de comunicação, transparência, compartilhamento de responsabilidades e alinhamento entre equipes.

Por fim, o artigo *Why are many businesses instilling a DevOps culture into their organization?* mostra os principais problemas que levam empresas a adotar DevOps, como excesso de tempo para liberar versões, falta de colaboração, silos organizacionais e baixa eficiência dos processos. Também apresenta resultados esperados dessa adoção, como faster time-to-market, melhor qualidade de software e aumento de produtividade. Isso confirma que planejamento, organização e priorização não são elementos secundários, mas parte estrutural da transformação DevOps.

Em conjunto, esses estudos mostram que DevOps depende de uma base organizacional sólida. A automação e a velocidade só produzem bons resultados quando estão apoiadas em planejamento, requisitos claros, comunicação eficiente e mecanismos de feedback.

DevOps practices

Entre as práticas identificadas na literatura, uma das mais importantes é a cultura colaborativa entre desenvolvimento e operações. Essa prática reduz silos, melhora a comunicação e favorece o compartilhamento de responsabilidades. No contexto deste trabalho, isso é essencial porque a organização e a priorização de tarefas se tornam mais eficazes quando os diferentes atores do processo compartilham visão e objetivos.

Outra prática fortemente destacada é a continuous integration, acompanhada de continuous testing. Essas práticas permitem integrar mudanças com frequência e validar continuamente o que foi desenvolvido. Isso contribui para o planejamento porque reduz o acúmulo de problemas, facilita o acompanhamento do progresso e permite corrigir rapidamente desvios no fluxo de trabalho.

Também aparecem com força as práticas de continuous delivery e continuous deployment. Elas estão ligadas à entrega rápida e frequente de novas versões. Entretanto, a própria literatura mostra que essas práticas dependem de uma boa organização prévia das tarefas. Sem definição clara de prioridade, sem planejamento de sprint e sem coordenação entre áreas, a velocidade de entrega tende a gerar instabilidade em vez de ganho real de eficiência.

A prática de Infrastructure as Code e o configuration management também são recorrentes. Elas reduzem atrasos causados por configuração manual de ambientes e aumentam a padronização da infraestrutura. Para o tema do trabalho, isso é importante porque um planejamento bem feito perde força quando a execução é travada por gargalos operacionais e ambientes mal configurados.

O monitoramento contínuo e a medição contínua aparecem como práticas que geram feedback operacional. Esse feedback é fundamental para reorganizar tarefas, redefinir prioridades e corrigir problemas com base em dados reais. Em outras palavras, o planejamento em DevOps não é algo fechado no início do processo: ele deve ser constantemente alimentado pelas informações produzidas ao longo da operação.

Por fim, a literatura também enfatiza requirements management, traceability, sprint planning e effort estimation. Essas práticas se conectam diretamente ao tema deste trabalho, pois tratam justamente da definição, organização e acompanhamento das tarefas e de suas relações com o restante do fluxo.

Tools

As ferramentas identificadas na literatura podem ser agrupadas conforme a função que exercem no ciclo DevOps.

No campo do planejamento e gerenciamento de tarefas, destacam-se Jira Software, Confluence e Azure DevOps. Essas ferramentas ajudam na organização do backlog,

na documentação de requisitos, no acompanhamento das tarefas e na visibilidade compartilhada do trabalho. Para o tema deste relatório, elas são especialmente importantes porque sustentam a organização e a priorização das atividades.

No campo do apoio ao planejamento e à tomada de decisão, o estudo sobre sprint planning menciona Planning Poker, um Naive Bayes classifier e um modelo de análise de sentimento. Esses recursos ajudam na estimativa de esforço, no suporte a decisões rápidas e no acompanhamento de sinais relacionados à comunicação e à maturidade do projeto.

No núcleo técnico do fluxo DevOps, aparecem ferramentas como GoCD, TeamCity, Octopus Deploy e GitHub, associadas à integração, entrega e deploy contínuos. Essas ferramentas tornam o fluxo mais automatizado, repetível e estável, conectando o planejamento inicial à execução prática do trabalho.

Na área de infraestrutura e ambientes, a literatura destaca Terraform e abordagens de Infrastructure as Code. Essas soluções automatizam o provisionamento e a configuração, diminuindo atrasos e gargalos operacionais.

Na parte de monitoramento e observabilidade, surgem Datadog e New Relic, que ajudam a acompanhar desempenho, falhas e comportamento do sistema. Essas ferramentas são importantes porque o monitoramento gera feedback para reorganizar o trabalho e ajustar prioridades futuras.

Para colaboração e comunicação, aparecem Slack e Hip Chat, cuja função principal é reduzir barreiras entre equipes e facilitar a troca rápida de informações. Já na área de testes e qualidade, a literatura menciona Selenium e Cucumber, que contribuem para validação contínua e maior confiabilidade do processo de entrega.

Assim, as ferramentas não aparecem apenas como suporte técnico, mas como elementos que sustentam planejamento, coordenação, automação, monitoramento e colaboração.

Pipeline

Com base na literatura selecionada e no foco do trabalho, propõe-se o seguinte pipeline conceitual:

Requirements and Prioritization -> Sprint Planning -> Task Organization -> Development -> Build -> Test -> Integration -> Deployment -> Monitoring -> Feedback and Reprioritization

Esse pipeline foi adaptado ao tema do grupo e, por isso, começa antes da etapa de desenvolvimento. A ideia central é mostrar que, no caso deste trabalho, DevOps não

pode ser reduzido apenas ao trecho técnico entre código e deploy. Antes disso, existe um momento essencial de levantamento de requisitos, definição de prioridades, planejamento do sprint e organização das tarefas.

A etapa de **Requirements and Prioritization** reúne as demandas e define quais tarefas devem receber maior prioridade. Em seguida, em **Sprint Planning**, essas prioridades são convertidas em um plano de execução. Depois, em **Task Organization**, as tarefas são distribuídas, estruturadas e acompanhadas.

Somente após essas etapas o fluxo entra em **Development, Build, Test, Integration e Deployment**, que correspondem às fases mais tradicionais do pipeline DevOps. Por fim, **Monitoring** produz informações sobre desempenho, falhas e comportamento do sistema, alimentando a etapa de **Feedback and Reprioritization**, que fecha o ciclo e permite novos ajustes no planejamento.

Esse pipeline reforça a principal tese do relatório: em DevOps, a qualidade do fluxo técnico depende de uma base inicial sólida de organização e priorização das tarefas, além de feedback contínuo para manter o processo adaptável.